

Paralelismo a nivel de hilos: Paralelización mediante OpenMP y programación asíncrona del análisis forense de manipulación de imágenes digitales

Lucía Monteagudo Castellanos¹[04656523-N] and Sara Pérez Peraza²[78821343-K]

¹ lmc146@alu.ua.es

² smpp4@alu.ua.es

Enlace al repositorio de github: <https://github.com/Luciamcs/Paralelismo-a-nivel-de-hilos.git>

Abstract. Esta práctica estudia la paralelización a nivel de hilos mediante OpenMP y `std::async` en C++. Primero se analizan conceptos básicos como variables compartidas y privadas, planificación de bucles, ejecución asíncrona y uso de `std::vector`. Después, estos conocimientos se aplican a un problema de análisis forense de imágenes digitales, donde se buscan partes paralelizables en la versión secuencial. Finalmente, se comparan distintas implementaciones mediante tiempos de ejecución, speed-up y eficiencia para determinar la estrategia de paralelización más adecuada.

Keywords: OpenMP, `std::async`, paralelismo, programación asíncrona, memoria compartida, análisis forense de imágenes, manipulación digital, ELA, DCT, speed-up.

1 Entrenamiento previo

1.1 Entrenamiento previo OpenMP

En este apartado se nos presenta el siguiente código:

```
#include <omp.h>
#define N 1000
#define CHUNKSIZE 100
main(int argc, char *argv[]) {
    int i, chunk;
    float a[N], b[N], c[N];
    /* Inicializamos los vectores */
    for (i=0; i < N; i++)
        a[i] = b[i] = i * 1.0;
    chunk = CHUNKSIZE;
    #pragma omp parallel shared(a,b,c,chunk) private(i)
```

```

{
#pragma omp for schedule(dynamic,chunk) nowait
for (i=0; i < N; i++)
c[i] = a[i] + b[i];
} /* end of parallel region */
}

```

Este código muestra un ejemplo de paralelización con OpenMP en el que se realiza la suma de dos vectores.

Primero, se incluyen las librerías necesarias y se definen dos constantes: el tamaño de los vectores ($N=1000$) y el tamaño del bloque (CHUNKSIZE). A continuación, en la función principal, se declaran tres vectores de tipo float: a,b y c, junto con las variables auxiliares i y chunk.[1]

A continuación, se inicializan los vectores a y b mediante un bloque secuencial, asignado a cada posición el valor de su índice. De esta forma, ambos vectores contienen los mismos valores.[1]

Después, se asigna el valor de CHUNKSIZE a la variable chunk, que posteriormente se utilizará en la planificación del trabajo entre hilos.[1]

Definimos la región paralela con la directiva:

```
#pragma omp parallel shared(a,b,c,chunk) private(i)
```

En este punto, OpenMp crea un conjunto de hilos que ejecutan el Código de forma concurrente. Las variables a,b,c y chunk se comparten entre todos los hilos, mientras que la variable i es privada para cada uno, evitando interferencias durante la ejecución del bucle.[1]

Dentro de esta región se utiliza la directiva:

```
#pragma omp for schedule(dynamic,chunk) nowait
```

Esta directiva sirve para repetir las iteraciones del bucle entre los hilos disponibles. Cada hilo ejecuta una parte del bucle, realizando la operación:

$$c[i] = a[i] + b[i];$$

De esta forma, el trabajo se divide entre los hilos, pero todos colaboran sobre los mismos vectores en memoria. Además, la cláusula nowait Evita que los hilos tengan que esperar al finalizar el bucle, lo que puede mejorar ligeramente el rendimiento.[1]

Finalmente, al salir de la región paralela, todos los hilos terminan su ejecución y el programa continua de forma secuencial.

1.1.1 ¿Para qué sirve la variable chunk?

La variable chunk se utiliza en la cláusula:

```
schedule(dynamic, chunk)
```

Sirve para definir el tamaño del bloque de iteraciones que se asigna a cada hilo en cada reparto de trabajo.[2]

En OpenMP, cuando se paraleliza un bucle con `#pragma omp for`, las iteraciones del bucle deben repartirse entre los distintos hilos. Este reparto no se hace necesariamente una a una (iteración a iteración), sino en grupos de iteraciones, llamados chunks. El valor de chunk indica cuántas iteraciones contiene cada uno de esos grupos.[2]

En este caso, como `CHUNKSIZE = 100`, cada hilo recibe bloques de 100 iteraciones. Por ejemplo:

El Bloque1 ejecuta las iteraciones 0–99
El bloque 2 ejecuta 100–199
etc.

Al usar `schedule(dynamic, chunk)`, estos bloques no se asignan todos al principio, sino que se reparten dinámicamente durante la ejecución:

1. Cada hilo toma un bloque de tamaño chunk.
2. Cuando termina ese bloque, solicita otro.
3. El proceso continúa hasta completar todas las iteraciones.

La utilidad de chunk es controlar la granularidad del reparto de trabajo. Un tamaño de bloque demasiado pequeño puede aumentar la sobrecarga de planificación, mientras que un tamaño demasiado grande puede provocar un reparto menos equilibrado entre hilos.[2]

1.1.2. Explica completamente el programa:

```
#pragma omp parallel shared(a,b,c,chunk) private(i)
```

¿Por qué y para qué se usa `shared(a,b,c,chunk)` en este programa?

Las variables `a`, `b`, `c` y `chunk` se declaran como compartidas (`shared`), lo que implica que todos los hilos creados en la región paralela acceden a una única copia de estas variables en memoria. Es decir, no se generan copias independientes para cada hilo, sino que todos trabajan sobre las mismas direcciones de memoria, dando coherencia al programa.[3][4]

En este caso, los vectores `a` y `b` actúan como datos de entrada que han sido previamente inicializados antes de la región paralela. Durante la ejecución paralela, los hilos leen sus valores para realizar el cálculo. Por este motivo, estas variables tienen que ser compartidas, ya que, si fueran privadas, cada hilo tendría su propia copia no inicializada, lo que daría lugar a resultados incorrectos o indefinidos.[3][4]

Por otro lado, el vector `c` es el encargado de almacenar el resultado de la suma de los vectores `a` y `b`. Aunque `c` es una variable compartida, no se producen conflictos entre hilos porque cada uno opera sobre índices distintos del vector. Gracias a la distribución de las iteraciones del bucle, cada hilo escribe en una porción diferente de `c`, evitando así solapamientos y condiciones de carrera. Esto permite que todos los hilos contribuyan de forma segura a la construcción de un único resultado global.[1]

La variable `chunk`, define el tamaño de los bloques de iteraciones que se asignan dinámicamente a los hilos mediante la cláusula `schedule(dynamic, chunk)` [2]. Es necesario que esta variable sea compartida para que todos los hilos sigan la misma estrategia de planificación del trabajo. Si `chunk` fuera privado, cada hilo podría utilizar valores distintos o incluso no inicializados, lo que provocaría un comportamiento incoherente en la distribución de las iteraciones.[2][4]

En conclusión, el uso de variables compartidas en este contexto permite que todos los hilos cooperen sobre un mismo conjunto de datos en memoria, haciendo posible la obtención de un resultado final coherente. No obstante, este enfoque requiere garantizar que cada hilo acceda a regiones independientes de escritura, como ocurre en este caso, para evitar problemas de sincronización como las condiciones de carrera.[2][3][4]

¿Por qué la variable `i` está etiquetada como `private` en el pragma?

La variable `i` se declara como privada (`private`) dentro del `pragma` de OpenMP para garantizar que cada hilo disponga de su propia copia independiente de esta variable durante la ejecución paralela. En el contexto del programa, `i` actúa como el índice del bucle que recorre los elementos de los vectores, por lo que es fundamental para asegurar un comportamiento adecuado del paralelismo.[4]

En una región paralela, si una variable no se especifica explícitamente como privada, puede ser compartida por todos los hilos. En el caso de la variable `i`, esto supondría que múltiples hilos intentarían leer y modificar simultáneamente la misma posición de memoria, lo que daría lugar a una condición de carrera (`race condition`). Este tipo de situación provoca resultados impredecibles, ya que el valor de `i` podría cambiar de forma no controlada dependiendo del orden de ejecución de los hilos.[4]

Al declararla como `private`, OpenMP crea automáticamente una copia independiente de la variable `i` para cada hilo. De este modo, cada hilo mantiene su propio valor del índice y lo actualiza conforme ejecuta las iteraciones del bucle que le han sido asignadas. Esto elimina cualquier interferencia entre hilos y asegura que cada uno procese correctamente su conjunto de iteraciones.[4]

Además, en el caso de un bucle paralelizado con la directiva `#pragma omp for`, OpenMP gestiona internamente la distribución de las iteraciones entre los hilos, asignando a cada uno un subconjunto de valores de `i`. Para que este mecanismo funcione correctamente, es importante que cada hilo tenga su propio contador de iteración, lo cual se consigue mediante la declaración de `i` como variable privada.[4]

En definitiva, el uso de `private(i)` es esencial para evitar condiciones de carrera y garantizar la correcta ejecución del bucle en paralelo, permitiendo que cada hilo opere de manera independiente sobre su parte de trabajo sin interferencias con el resto.[4]

1.1.3. ¿Para qué sirve `Schedule`? ¿Qué otras posibilidades hay?

La cláusula `schedule` en OpenMP se utiliza para definir la estrategia mediante la cual se reparten las iteraciones de un bucle entre los distintos hilos que participan en una región paralela. Su objetivo principal es controlar cómo se distribuye la carga de trabajo, lo cual influye directamente en el rendimiento y la eficiencia del programa paralelo.[2]

En el código proporcionado se emplea la directiva `schedule(dynamic, chunk)`, lo que indica que las iteraciones del bucle se dividen en bloques de tamaño `chunk` y se asignan dinámicamente a los hilos durante la ejecución. En este tipo de planificación, cada hilo solicita un bloque de iteraciones cuando termina el anterior, lo que permite un mejor equilibrio de carga (carga balanceada), especialmente en situaciones donde el coste de las iteraciones no es uniforme. De este modo, los hilos que terminan antes continúan recibiendo trabajo, evitando que queden inactivos mientras otros siguen procesando.[2]

Sin embargo, la planificación dinámica también introduce cierta sobrecarga debido a la gestión continua de la asignación de bloques, ya que el sistema debe coordinar qué hilo recibe el siguiente conjunto de iteraciones. Por tanto, su uso es más adecuado cuando existe un desequilibrio potencial en el tiempo de ejecución de las iteraciones.[2]

Además del tipo `dynamic`, OpenMP ofrece otras políticas de planificación que pueden ser más adecuadas dependiendo del problema:

- `schedule(static)`: reparte las iteraciones de forma fija entre los hilos desde el inicio. Tiene muy baja sobrecarga y es eficiente cuando todas las iteraciones tienen un coste similar. [2]
- `schedule(dynamic)`: asigna bloques de iteraciones de forma dinámica durante la ejecución, mejorando el equilibrio de carga a costa de mayor sobrecarga. [2]
- `schedule(guided)`: similar a la dinámica, pero comienza con bloques grandes que se van reduciendo progresivamente. [2]
- `schedule(auto)`: deja que el compilador o entorno de ejecución decida la estrategia. [2]
- `schedule(runtime)`: permite definir la planificación en tiempo de ejecución mediante variables de entorno.[2]

En conclusión, la cláusula `schedule` es un mecanismo clave en OpenMP para optimizar la distribución del trabajo entre hilos. La elección de una u otra estrategia depende de las características del problema, especialmente de la uniformidad del coste de las iteraciones, siendo fundamental para alcanzar un buen rendimiento en aplicaciones paralelas.[2]

1.1.4 ¿Qué tiempos y otras medidas de rendimiento podemos medir en secciones de código paralelizadas con OpenMP?

En secciones de código paralelizadas con OpenMP se pueden medir distintos tiempos y métricas de rendimiento que permiten analizar si el paralelismo está funcionando de forma eficiente. La medida más básica es el tiempo de ejecución, que indica cuánto tarda en completarse una parte del programa. Este tiempo puede obtenerse fácilmente utilizando funciones como `omp_get_wtime()`, que devuelve el tiempo en segundos.[5]

```
double start = omp_get_wtime();

#pragma omp parallel shared(a,b,c,chunk) private(i)
{
    #pragma omp for schedule(static, chunk) nowait
    for (i = 0; i < N; i++)
```

```
c[i] = a[i] + b[i];
}
double end = omp_get_wtime();
```

A partir de este tiempo se pueden calcular otras métricas importantes. Una de ellas es el speed-up o ganancia en velocidad, que compara el tiempo de ejecución del programa en versión secuencial con el tiempo en versión paralela. Esta medida permite ver cuánto mejora el rendimiento al utilizar varios hilos. En un caso ideal, el speed-up sería igual al número de hilos, aunque en la práctica esto no suele cumplirse completamente.[6]

$$\text{Speed-up} = T_{\text{secuencial}} / T_{\text{paralelo}}$$

Otra métrica relevante es la eficiencia, que se obtiene dividiendo el speed-up entre el número de hilos utilizados. Esto indica cómo de bien se están aprovechando los recursos disponibles. Si la eficiencia es alta, significa que el paralelismo está siendo efectivo, mientras que si es baja puede indicar problemas como sobrecarga o mal reparto del trabajo.[6]

$$\text{Eficiencia} = \text{Speed-up} / \text{Número de hilos}$$

A continuación, veremos una tabla con el número de hilos, tiempo de ejecución, speed-up y eficiencia que hemos obtenido del código de la practica si ponemos diferentes números de hilos y cambiamos la N por un número superior (ya que si no nos sale tiempos muy pequeños):

Nº de hilos	Tiempo de ejecución	Speed-up	Eficiencia
1	0.00024825 s	1.00	1.00
2	0.000429212 s	0.58	0.29
4	0.000359601 s	0.69	0.17

También es importante tener en cuenta el reparto de la carga entre hilos. Si algunos hilos terminan antes que otros, el rendimiento no será óptimo. Esto depende en gran medida de cómo se distribuyen las iteraciones del bucle, por ejemplo, mediante la cláusula schedule.

Por último, se puede analizar cómo cambia el rendimiento al aumentar el número de hilos o el tamaño del problema, lo que se conoce como escalabilidad. Esto permite ver

hasta qué punto el programa se beneficia del paralelismo o si hay limitaciones, como partes del código que no se pueden paralelizar o problemas de acceso a memoria.[6]

En general, todas estas medidas ayudan a entender mejor el comportamiento del programa paralelo y a detectar posibles mejoras para hacerlo más eficiente.

1.2 Entrenamiento previo `std::async`

Observa el siguiente programa en c++ donde se llama a dos funciones con `std::async`:

```
#include <iostream>
#include <future>
#include <chrono>

int task(int id, int millis) {
    std::this_thread::sleep_for(std::chrono::milliseconds(millis));
    std::cout<<"Task "<<id<<" completed"<<std::endl;
    return id;
}

int main() {
    auto start = std::chrono::high_resolution_clock::now();
    std::future<int> task1 = std::async(std::launch::async,
    task, 1, 2000);
    std::future<int> task2 = std::async(std::launch::async,
    task, 2, 3000);

    task1.wait();
    int taskId = task2.get();

    auto end = std::chrono::high_resolution_clock::now();
    auto elapsed
    std::chrono::duration_cast<std::chrono::milliseconds>(end-start);

    std::cout<<"Completed in: "<<elapsed.count()<<"ms"<<std::endl;
}
```

1.2.1. ¿Para qué sirve el parámetro `std::launch::async`?

El parámetro `std::launch::async` se utiliza para indicar que la función pasada a `std::async` debe ejecutarse de forma asíncrona, es decir, en un hilo separado del hilo

principal. De esta manera, la tarea comienza su ejecución en paralelo tan pronto como se invoca `std::async`, sin esperar a que el programa principal termine otras instrucciones.[7]

En el código propuesto, este parámetro permite que las dos llamadas a la función `task` se lancen de manera concurrente. Así, ambas tareas pueden ejecutarse al mismo tiempo, aprovechando el paralelismo a nivel de hilos. Esto hace posible reducir el tiempo total de ejecución respecto a una ejecución secuencial, ya que el programa no espera a que una tarea termine para iniciar la otra.[7]

Además, `std::launch::async` garantiza que realmente se cree ejecución asíncrona. Esto es importante porque, si no se especifica ninguna política de lanzamiento, el sistema puede decidir ejecutar la tarea de forma diferida (`deferred`), es decir, no lanzarla inmediatamente, sino esperar a que se invoque `wait()` o `get()` sobre el objeto `future`. [7]

Por tanto, `std::launch::async` sirve para forzar que la tarea se ejecute en paralelo desde el momento de su creación, permitiendo explotar el paralelismo y mejorar el rendimiento cuando las tareas son independientes entre sí.[7]

1.2.2 Calcula el tiempo que tarda el programa con `std::launch::async` y `std::launch::deferred` ¿A qué se debe la diferencia de tiempos?

Con `std::launch::async`, las dos tareas se lanzan de forma asíncrona y pueden ejecutarse en paralelo.[7] Como una tarea tarda 2000 ms y la otra 3000 ms, el tiempo total será aproximadamente el de la tarea más larga, es decir, unos 3000 ms. Esto se realiza ya que tenemos estas dos líneas en el código:

```
std::future<int> task1 = std::async(std::launch::async,
task, 1, 2000);

std::future<int> task2 = std::async(std::launch::async,
task, 2, 3000);
```

El resultado tras la ejecución es el siguiente:

```
Lucia@lucia-Thin-A15-87VE:~/Escritorio/SE_Cuatri_2/practica_4_car/sourceP3/source/Task0$ make run_async
g++ -fopenmp -lpthread async_functions.cpp -o async_functions
./async_functions
Task 1 completed after 2000 milliseconds.
Task 2 completed after 3000 milliseconds.
All tasks completed in 3000 milliseconds.
```

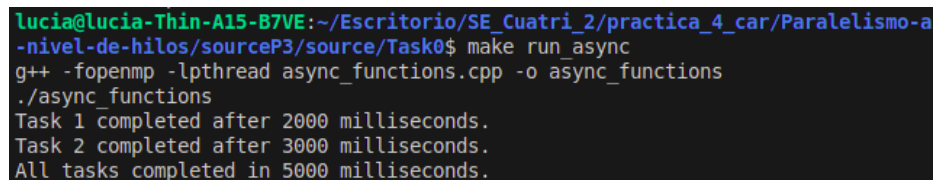
Con `std::launch::deferred`, las tareas no se ejecutan al crearse, sino cuando se llama a `wait()` o `get()`. En este caso primero se ejecuta `task1.wait()`, que tarda 2000 ms, y después `task2.get()`, que tarda 3000 ms. Por tanto, el tiempo total será 5000 ms. Para

sacar estos datos hemos cambiado las dos líneas anteriores por estas, para que se ejecute con `std::launch::deferred`:

```
std::future<int> task1 = std::async(std::launch::deferred, task, 1, 2000);

std::future<int> task2 = std::async(std::launch::deferred, task, 2, 3000);
```

Su resultado fue el siguiente:



```
lucia@lucia-Thin-A15-B7VE:~/Escritorio/SE_Cuatri_2/practica_4_car/Paralelismo-a
-nivel-de-hilos/sourceP3/source/Task0$ make run_async
g++ -fopenmp -lpthread async_functions.cpp -o async_functions
./async_functions
Task 1 completed after 2000 milliseconds.
Task 2 completed after 3000 milliseconds.
All tasks completed in 5000 milliseconds.
```

La diferencia se debe a que `std::launch::async` permite ejecución paralela en distintos hilos, mientras que `std::launch::deferred` aplaza la ejecución y en este programa provoca una ejecución secuencial.[7]

1.2.3 ¿Qué diferencia hay entre los métodos `wait` y `get` de `std::future`?

El método `wait()` de `std::future` bloquea la ejecución del hilo actual hasta que la tarea asociada al future ha terminado, pero no obtiene el valor devuelto por dicha tarea. [8] En cambio, el método `get()` también bloquea hasta que la tarea termina, pero además devuelve el resultado producido por la función ejecutada de forma asíncrona.[9]

En el programa de ejemplo se utilizan ambos métodos:

```
task1.wait();

int taskId = task2.get();
```

En este caso, `task1.wait()` espera a que finalice la primera tarea, pero no recupera su valor de retorno. Sin embargo, `task2.get()` espera a que finalice la segunda tarea y guarda el valor devuelto en la variable `taskId`. [8] Además, `get()` solo puede llamarse una vez, ya que consume el resultado almacenado en el future.[9]

1.2.4 ¿Qué ventajas ofrece `std::async` frente a `std::thread`?

`std::async` ofrece varias ventajas frente a `std::thread`. La principal es que simplifica la ejecución de tareas asíncronas, ya que devuelve un objeto `std::future` que permite esperar a que la tarea termine y obtener directamente el valor devuelto por la función mediante `get()`. [7], [9]

Por ejemplo:

```
std::future<int> resultado = std::async(std::lau-
nch::async, task, 1, 2000);

int valor = resultado.get();
```

En este caso, la función `task` se ejecuta de forma asíncrona y su valor de retorno se obtiene fácilmente mediante `get()`. Con `std::thread`, obtener un valor devuelto por una función es más complejo, ya que sería necesario usar variables compartidas, referencias, `std::promise` u otros mecanismos de sincronización. [7], [9], [10]

Además, `std::async` facilita la propagación de excepciones, porque si la tarea genera una excepción, esta se almacena en el `future` y se relanza al llamar a `get()`. También evita parte de la gestión manual de hilos que requiere `std::thread`, como llamar explícitamente a `join()` o `detach()`. [10]

Otra ventaja es que permite indicar la política de ejecución mediante parámetros como `std::launch::async` o `std::launch::deferred`, pudiendo decidir si la tarea se ejecuta en paralelo o si se aplaza hasta que se necesite el resultado. Por tanto, `std::async` es más cómodo, seguro y adecuado cuando se quiere ejecutar una función en paralelo y obtener su resultado. [7], [9], [10]

1.3 Entrenamiento previo `std::vector`

Observa el siguiente programa en `c++` donde se inicializa un vector `std::vector` y se rellena con valores:

```
#include <vector>
#include <iostream>
#include <chrono>
int main() {
    // default initialization and add elements with push_back
    auto start = std::chrono::high_resolution_clock::now();

    std::vector<float> v1;
    for (int i = 0; i < 10000; i++)
```

```

v1.push_back(i);

auto end = std::chrono::high_resolution_clock::now();
auto
elapsed
=
std::chrono::duration_cast<std::chrono::millise-
conds>(end-start);
std::cout<<"Default initialization:
"<<elapsed.count()<<"ms"<<std::endl;

// initialized with required size and add elements with
direct access
start = std::chrono::high_resolution_clock::now();

std::vector<float> v2(10000);
for (int i = 0; i < 10000; i++)
v2[i] = i;

end = std::chrono::high_resolution_clock::now();

elapsed
=
std::chrono::duration_cast<std::chrono::millise-
conds>(end-start);
std::cout<<"Initialization with
size:
"<<elapsed.count()<<"ms"<<std::endl;
}

```

Responde a las siguientes preguntas:

1.3.1 ¿Cuál de las dos formas de inicializar el vector y rellenarlo es más eficiente? ¿Por qué?

En el programa se comparan dos formas de inicializar y rellenar un vector. La primera consiste en crear el vector vacío e insertar elementos mediante `push_back()`:

```

std::vector<float> v1;

for (int i = 0; i < 10000; i++)
    v1.push_back(i);

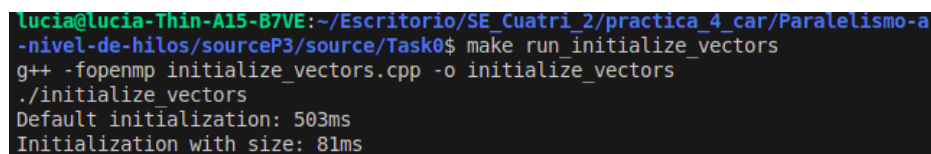
```

La segunda forma consiste en crear el vector directamente con el tamaño necesario y después asignar los valores mediante acceso por índice:

```
std::vector<float> v2(10000);

for (int i = 0; i < 10000; i++)
    v2[i] = i;
```

Los tiempos obtenidos fueron los siguientes:



```
lucia@lucia-thin-A15-B7VE:~/Escritorio/SE_Cuatri_2/practica_4_car/Paralelismo-a
-nivel-de-hilos/sourceP3/source/Task0$ make run_initialize_vectors
g++ -fopenmp initialize_vectors.cpp -o initialize_vectors
./initialize_vectors
Default initialization: 503ms
Initialization with size: 81ms
```

La forma más eficiente es inicializar el vector con el tamaño necesario y después acceder directamente a sus posiciones, es decir la siguiente forma:

```
std::vector<float> v2(10000);
for (int i = 0; i < 10000; i++)
    v2[i] = i;
```

Esto es más eficiente porque el vector reserva desde el principio la memoria necesaria para almacenar los 10000 elementos. De esta forma, el bucle solo tiene que asignar valores a posiciones ya existentes.

En cambio, si se crea el vector vacío y se añaden elementos con `push_back`, el vector puede necesitar varias realocaciones de memoria a medida que crece. En cada realocación se reserva una nueva zona de memoria y se copian o mueven los elementos existentes, lo que añade sobrecoste. [12]

Por eso, para un tamaño conocido de antemano, es preferible inicializar el vector con ese tamaño.

1.3.2 ¿Podría ocurrir algún problema al paralelizar los dos bucles for? ¿Por qué?

Sí, podría haber problemas al paralelizar los bucles, sobre todo en el primer caso, donde se usa `push_back()` sobre un vector inicialmente vacío.[12]

```
std::vector<float> v1;
```

```
#pragma omp parallel for
for (int i = 0; i < N; i++)
    v1.push_back(i);
```

Esta operación modifica el tamaño interno del vector y puede provocar realocaciones de memoria. Si varios hilos ejecutan `push_back()` al mismo tiempo sobre el mismo vector, se pueden producir condiciones de carrera, corrupción de memoria o resultados incorrectos.[12]

En cambio, el segundo bucle es más adecuado para paralelizar:

```
std::vector<float> v2(N);
#pragma omp parallel for
for (int i = 0; i < N; i++)
    v2[i] = i;
```

En este caso, el vector se crea previamente con el tamaño necesario y cada iteración escribe en una posición distinta mediante `v2[i] = i`. Como no se modifica el tamaño del vector durante el bucle y no hay dos hilos escribiendo en la misma posición, este caso puede paralelizarse de forma segura con OpenMP.[11],[12]

Por tanto, el problema principal no está en paralelizar un bucle sobre un vector, sino en paralelizar operaciones que modifican dinámicamente la estructura interna del contenedor, como ocurre con `push_back()`.[12]

2 Paralelización del análisis forense de manipulación de imágenes digitales

En este apartado nos vamos a centrar en la paralelización del análisis forense de manipulación de imágenes digitales.

Para ello, se parte de una solución secuencial que aplica distintas técnicas de análisis, como el estudio del patrón de ruido, el Error Level Analysis y la Transformada Discreta del Coseno, con el objetivo de detectar posibles regiones manipuladas en una imagen.

A partir de este código inicial, se deberán identificar los procesos principales, analizar sus dependencias y tiempos de ejecución, y aplicar estrategias de paralelización mediante OpenMP y programación asíncrona con `std::async`, buscando mejorar el rendimiento y aprovechar los núcleos disponibles del sistema.

2.1 Analiza el código e identifica los distintos procesos

En la primera parte de esta tarea, debemos analizar el código de la solución secuencial que se proporciona para identificar los distintos procesos que se aplican sobre la imagen de entrada, así como las dependencias entre ellos y las partes que podrían ejecutarse en paralelo.

El análisis comienza examinando el archivo `main.cc` que contiene la función principal del programa y define el flujo general de ejecución. Desde el `main`, se llaman a funciones definidas para el procesamiento de imágenes: `compute_srm`, `compute_ela` y `compute_dct`. Estas funciones implementan los distintos análisis forenses mencionados en el enunciado de la práctica: SRM, ELA y DCT. También, hay que identificar los procesos aplicados sobre la imagen manipulada y elaborar un diagrama de dependencias, además de señalar qué procesos podrían ejecutarse en paralelo.

Lo primero que se ejecuta es la función `main`, por lo que, dentro de ella, vamos a identificar los principales procesos que se dan, así como las dependencias y llamadas a otras funciones. La primera parte es:

```
if(argc == 1) {
    std::cerr<<"Image filename missing from arguments.
Usage ./dct <filename>"<<std::endl;
    exit(1);
}
```

Básicamente aquí se comienza comprobando que se ha introducido una imagen como argumento, si no se añade, saltará el error definido. Si se añade la imagen, pasaremos al siguiente bloque:

```
int block_size=8;
```

Que fija el tamaño de bloque para DCT. Posteriormente:

```
Image<unsigned char> image = load_from_file(argv[1]);
```

Cargamos la imagen que vamos a utilizar, este proceso es obligatorio y debe ejecutarse antes que los demás, porque todos los análisis posteriores usan la variable `image`. Aquí encontramos la primera dependencia, además, sabemos que este proceso no puede ejecutarse en paralelo con los análisis, porque evidentemente, los análisis dependen de que la imagen ya esté cargada. Finalmente:

```
Image<unsigned char> srm3x3 = compute_srm(image, 3);
save_to_file("srm_kernel_3x3.png", srm3x3);
```

```

    save_to_file("srm_kernel_5x5.png", compute_srm(image,
5));
    save_to_file("ela.png", compute_ela(image, 90));
    save_to_file("dct_invert.png", compute_dct(image,
block_size, true));
    save_to_file("dct_direct.png", compute_dct(image,
block_size, false));

```

A partir de la imagen original se ejecutan 5 procesos:

- SRM 3x3: Es el primer proceso de este bloque que se ejecuta, aquí, `compute_srm`, realiza los siguientes pasos:

```

Image<float> srm = image.to_grayscale().con-
vert<float>();
srm = srm.convolution(get_srm_kernel(kernel_size));
srm = srm.abs().normalized();
srm = srm * 255;
Image<unsigned char> result = srm.convert<unsigned
char>();

```

Es decir, de la imagen original, hacemos la conversión a escala de grises y a su vez, hacemos su conversión a float, posteriormente, aplicamos convolución con un kernel de 3x3, obtenemos el valor absoluto resultante de la convolución, el cual se normaliza para posteriormente, multiplicarse por 255. Finalmente, hacemos la conversión a unsigned char y obtenemos la imagen resultante. Este proceso sirve para resaltar patrones de ruido de la imagen.

- SRM 5x5: Es igual al proceso anterior, con la única diferencia de que ahora aplicamos un kernel de 5x5. Internamente sigue los mismos pasos que el anterior solo que con un kernel de mayor tamaño, haciendo que este analice más píxeles vecinos, aumenta el coste computacional en comparación a SRM 3x3.
- ELA: ELA es el Error Level Analysis explicado en la práctica, se llama haciendo `compute_ela(image,90)`, realiza los siguientes pasos:

```

Image<unsigned char> grayscale = image.to_grayscale();
save_to_file("_temp.jpg", grayscale, quality);
Image<float> compressed =
load_from_file("_temp.jpg").convert<float>();
compressed = compressed + (grayscale.con-
vert<float>()*(-1));
compressed = compressed.abs().normalized() * 255;

```

```
Image<unsigned char> result = compressed.convert<unsigned char>();
```

Partimos de la imagen original, la cual se convierte primero a escala de grises, se hace un guardado temporal como jpg con la calidad especificada en la llamada, en este caso, sería 90. Posteriormente, se hace la lectura de la imagen jpg recomprimada, lo siguiente es restar entre la imagen recomprimada e imagen original en gris, esto recordemos que es para detectar las variaciones de ruido que se dan en los puntos donde se modifican las imágenes. Posteriormente, aplicamos el valor absoluto, normalizamos el resultado y multiplicamos por 255. Por último, se hace la conversión a unsigned char y obtenemos la imagen resultante ELA.

- DCT inversa: Se le llama con `compute_dct` y uno de los parámetros de la función es si se realiza de manera inversa o no, como en esta llamada se pone a true, sabemos que vamos a hacer una DCT inversa:

```
Image<float> grayscale = image.convert<float>().to_grayscale();
std::vector<Block<float>> blocks = grayscale.get_blocks(block_size);

for(int i=0;i<blocks.size();i++){
    float **dctBlock = dct::create_matrix(block_size, block_size);
    dct::direct(dctBlock, blocks[i], 0);
    if (invert) {
        for(int k=0;k<blocks[i].size/2;k++)
            for(int l=0;l<blocks[i].size/2;l++)
                dctBlock[k][l] = 0.0;
        dct::inverse(blocks[i], dctBlock, 0, 0.0, 255.);
    }else dct::assign(dctBlock, blocks[i], 0);
    dct::delete_matrix(dctBlock);
}
```

Primero, convertimos a float y posteriormente a escala de grises, posteriormente, dividimos en bloques de 8x8. Ahora dentro del bucle iteramos y para cada bloque: creamos una matriz auxiliar, se aplica la DCT directa, luego ponemos a 0 la parte de los coeficientes de la matriz, aplicamos la dct inversa y finalmente borramos la matriz auxiliar antes de la siguiente iteración. El último paso es la conversión a unsigned char y obtenemos el resultado de DCT inversa.

- DCT directa: Se ejecuta con `compute_dct` y en el parámetro de si queremos hacer DCT inversa o no, pondremos false. Internamente funciona igual que la inversa con la diferencia de que no se ejecuta el bloque condicional de si es inversa (nos ahorramos 2 bucles anidados, donde se colocan a 0 la parte de los

coeficientes de la matriz y se hace la dct inversa). El flujo es igual eliminando algunas cosas del anterior: primero, se hace la conversión de la imagen a float y posteriormente a escala de grises, luego se divide en bloques de 8x8 y para cada bloque, creamos una matriz auxiliar, aplicamos la DCT directa, asignamos los coeficientes DCT al bloque y finalmente eliminamos la matriz auxiliar. Por último, hacemos la conversión a unsigned char y obtenemos el resultado de DCT directa.

Ahora que conocemos el flujo de ejecución podemos repasar las dependencias de cada parte para posteriormente poder realizar el diagrama de dependencias. Lo primero antes de que podamos hacer ninguna llamada es cargar la imagen, si la imagen no se ha cargado, no podemos ejecutar ninguno de los bloques anteriores ya que todos trabajan sobre la imagen.

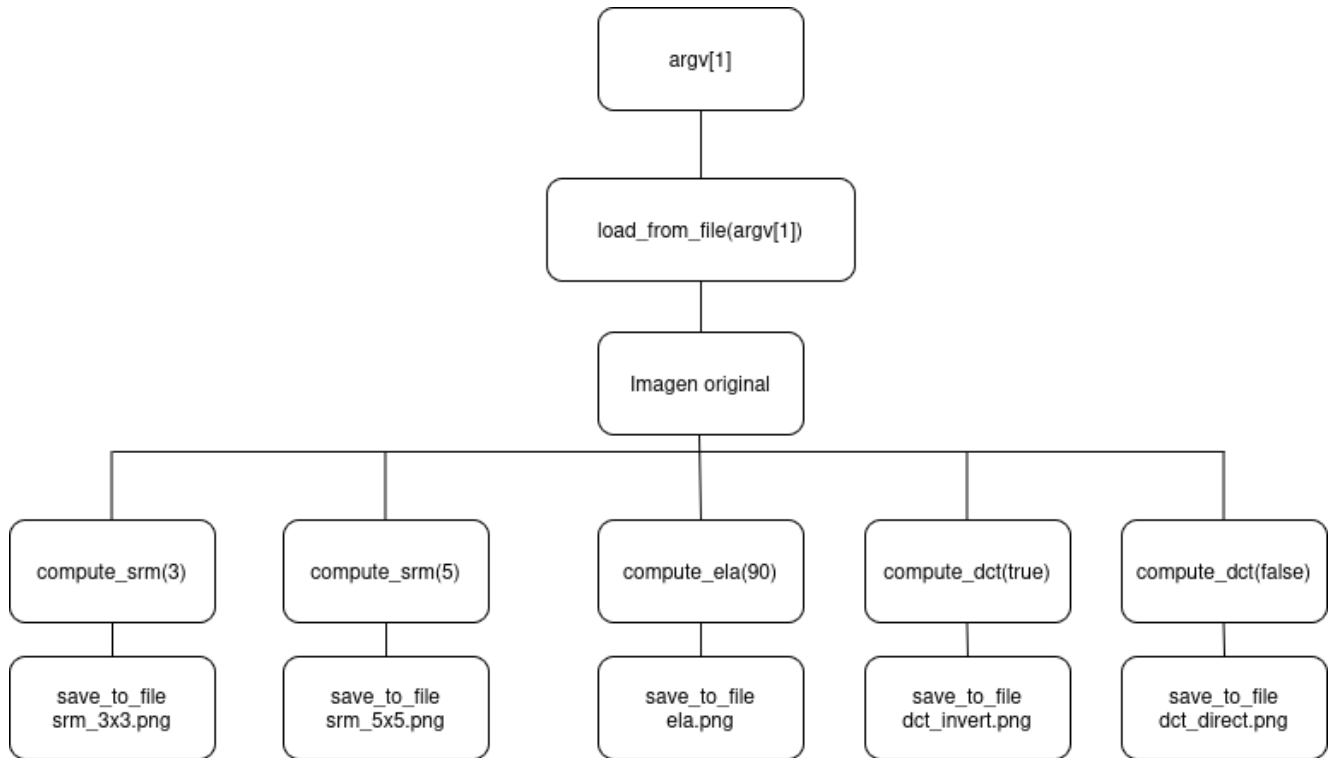
Una vez cargada la imagen, los procesos `compute_srm(image, 3)`, `compute_srm(image, 5)`, `compute_ela(image, 90)`, `compute_dct(image, block_size, true)` y `compute_dct(image, block_size, false)` no presentan dependencias entre sí. Todos utilizan la imagen original como entrada, pero generan resultados independientes. Esto permite identificar paralelismo funcional entre los distintos análisis.

Luego, cada operación de guardado mediante `save_to_file` depende únicamente del resultado generado por su proceso correspondiente, por ejemplo, `srm_kernel_3x3.png` depende del resultado de `compute_srm(image, 3)`, mientras que `dct_direct.png` depende del resultado de `compute_dct(image, block_size, false)`.

Para `compute_srm` y `compute_ela`, existe una dependencia secuencial (no entre ellas, sino para sí mismas), ya que los pasos dentro de cada una, normalmente depende del resultado justo anterior, por ejemplo, aplicamos la conversión a float una vez que ya tenemos la imagen en escala de grises, o solo podemos normalizar cuando ya hemos calculado el valor absoluto, por lo que internamente, las etapas generales deben ejecutarse secuencialmente. Para `compute_dct` sucede lo mismo, solo con el matiz de que la ejecución inversa y la directa no dependen entre sí, por lo que se pueden paralelizar perfectamente.

En resumen, el programa presenta una dependencia inicial común, la carga de la imagen, y posteriormente varios procesos independientes entre sí. Esto permite explotar paralelismo funcional entre análisis y paralelismo de datos dentro de algunos procesos, especialmente en el procesamiento por bloques de la DCT y en operaciones por píxel como la convolución, la resta, el valor absoluto y la normalización.

El diagrama realizado es el siguiente:

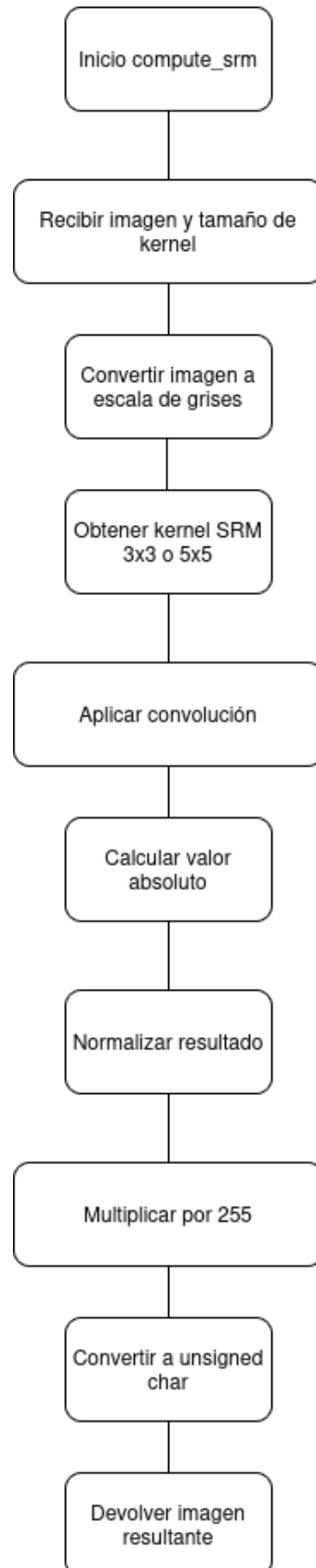


Primero se debe realizar la carga de la imagen obligatoriamente, ya que de la imagen es que dependen los 5 procesos, luego, cada proceso es independiente de los otros, por lo que se pueden ejecutar en paralelo, obteniendo los resultados de cada proceso.

2.2 Analiza el código de cada proceso y el tiempo

Como hemos visto en la tarea 2.1, los procesos identificados son los siguientes: carga de imagen, compute_srm(3), compute_srm(5), compute_ela, compute_dct(true), compute_dct(false) y guardar los resultados. Aquí nos vamos a centrar en compute_srm, compute_ela y compute_dct. Ya en la tarea 2.1, explicamos los pasos que sigue cada uno, por lo que aquí simplemente vamos a crear el diagrama de flujo de cada uno directamente, y debajo de cada diagrama, señalaremos las partes paralelizables.

Diagrama de flujo de compute_srm:

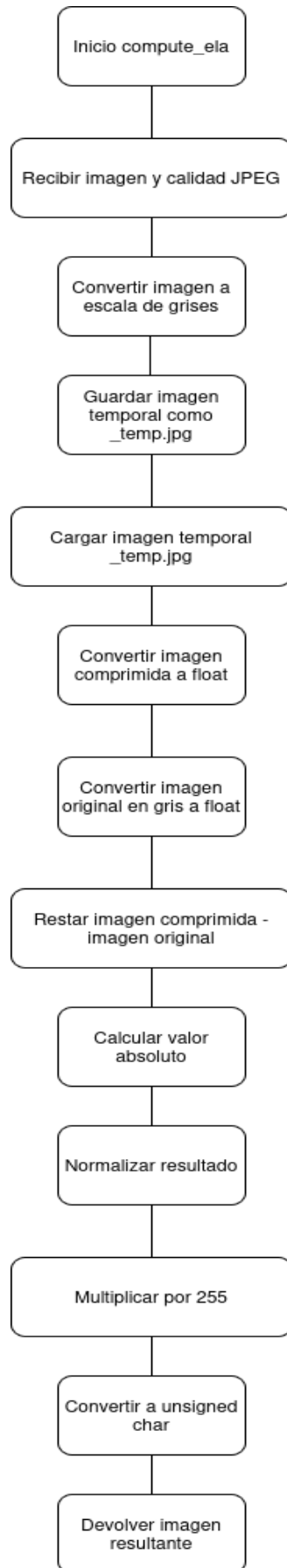


Las partes potencialmente paralelizables son aquellas operaciones que recorren la imagen píxel a píxel, estos son: conversión a escala de grises, conversión a float, convolución, valor absoluto, normalización, multiplicación por 255 y conversión a unsigned char. El punto más interesante es la convolución, porque para cada píxel de salida se aplica el kernel sobre una vecindad de píxeles. Normalmente, cada píxel de salida puede calcularse de forma independiente, por lo que sería un buen candidato para OpenMP.

Además, el SRM 3x3 y el SRM 5x5 son independientes entre sí, así que también se podrían ejecutar en paralelo como tareas separadas usando `std::async`.

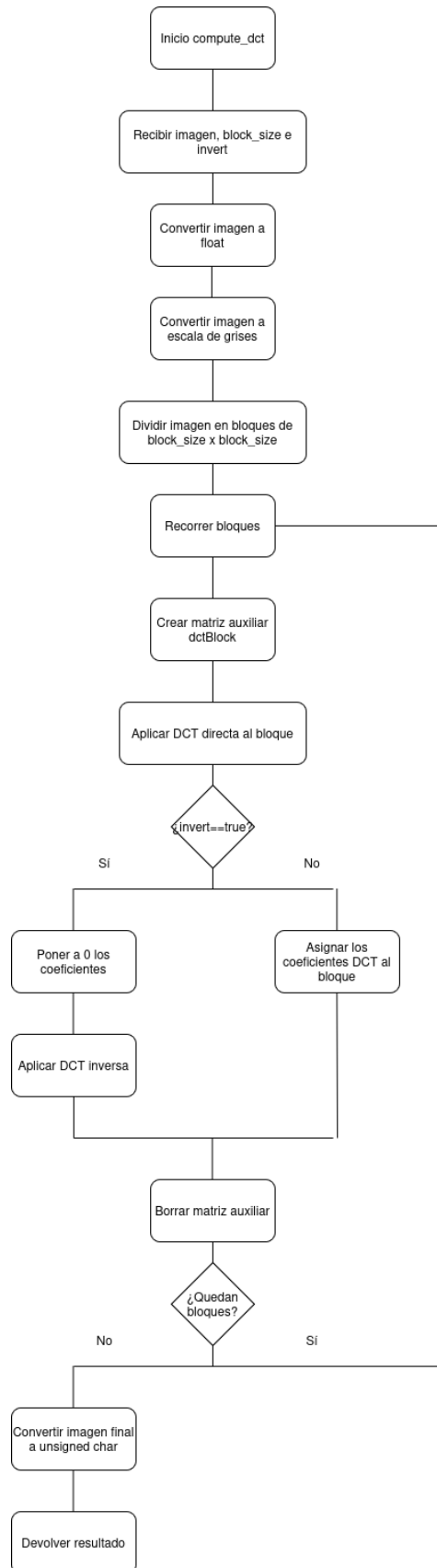
Ahora, el siguiente proceso es `compute_ela`, que es bastante similar al anterior, aunque aquí tenemos una dependencia clara dentro del propio código entre `save_to_file("_temp.jpg", grayscale, quality)` y `Image<float> compressed = load_from_file("_temp.jpg").convert<float>()`, ya que no podemos cargar `_temp.jpg` hasta que no la hayamos guardado previamente.

Diagrama de flujo:



Para `compute_ela` hay partes que son claramente secuenciales y que no vamos a poder paralelizar, esta parte es: Guardar `_temp.jpg` -> Cargar `_temp.jpg` ya que esa dependencia no se puede romper, porque primero debe existir el archivo temporal para poder cargarlo. Las partes que sí podrían paralelizarse son: conversión a escala de grises, conversión a float, resta entre imágenes, valor absoluto, normalización, multiplicación por 255 y conversión final a `unsigned_char`. Sin embargo, hay que tener cuidado con el archivo temporal `_temp.jpg`. Si se ejecutaran varias funciones `compute_ela` en paralelo, podrían pisarse entre sí porque todas usarían el mismo nombre de archivo. En este programa solo hay una llamada a `compute_ela`, así que no genera conflicto.

El último proceso es `compute_dct`, su diagrama de flujo es:



En `compute_dct` la parte más clara para la paralelización es el bucle, podemos crear varios hilos que repartan las iteraciones, pero, además, podríamos paralelizar las operaciones que recorren píxel a píxel la imagen como en los casos de las funciones anteriores, estas operaciones son: conversión a float, conversión a escala de grises, división en bloques y conversión final a unsigned char, aunque lo que nos va a generar más ganancia es la paralelización del bucle.

Para finalizar la tarea 2.2, nos piden calcular los tiempos que tardan cada una de las funciones de manera secuencial, por suerte, el código del profesor ya implementa los métodos necesarios para que la ejecución nos devuelva los tiempos, por lo que a continuación vamos a escribir los resultados.

Proceso	Parámetros	Tiempo secuencial
SRM 3x3	Kernel_size=3	259 ms
SRM 5x5	Kernel_size=5	691 ms
ELA	quality=90	207 ms
DCT Inversa	Block_size=8, invert=true	3398 ms
DCT Directa	Block_size=8, invert=false	2139ms

Los resultados de esta tabla se han obtenido tras la ejecución del makefile aportado:

```
Computing SRM 3x3...
SRM elapsed time: 259ms
Computing SRM 5x5...
SRM elapsed time: 691ms
Computing ELA...
ELA elapsed time: 207ms
Computing inverse DCT 8x8...
DCT elapsed time: 3398ms
Computing direct DCT 8x8...
DCT elapsed time: 2139ms
```

2.3 Paralelización

Los resultados que se han obtenidos tras paralelizar el código han sido los siguientes:

- Resultados con Async:

```

Tareas asíncronas
Computing SRM 5x5...
Computing SRM 3x3...
Computing ELA...
Computing inverse DCT 8x8...
Computing direct DCT 8x8...
ELA elapsed time: 227ms
SRM elapsed time: 356ms
SRM elapsed time: 716ms
DCT elapsed time: 2274ms
DCT elapsed time: 3360ms
Tiempo total de async: 3732ms

```

- Resultados con OpenMP:

```

Máximo de hilos openMP: 16
Computing SRM 3x3...
SRM elapsed time: 262ms
Computing SRM 5x5...
SRM elapsed time: 551ms
Computing ELA...
ELA elapsed time: 204ms
Computing DCT inverse DCT 8x8 with OpenMP...
DCT OpenMP elapsed time: 894ms
Computing DCT direct DCT 8x8 with OpenMP...
DCT OpenMP elapsed time: 428ms
Tiempo total OpenMP: 2723ms

```

Una vez terminado el código para paralelizar con async y con openMP, pasamos a responder las siguientes preguntas que se plantean en la práctica (la explicación del código se hará más adelante en el apartado de “Sobre el código implementado”):

(1) ¿Se obtiene un mejor rendimiento paralelizando todas las partes posibles?

No necesariamente, no siempre se obtiene mejor rendimiento paralelizando todas las partes posibles. En esta práctica, a pesar de que varios procesos son paralelizables, no

todos aportan la misma mejora. Si recordamos de apartados anteriores, el tiempo de ejecución de SRM 3x3 y de ELA son bastante escasos, mientras que el tiempo que tarda DCT es bastante más extenso, lo que hace que paralelizar SRM 3x3 o ELA produzca una mejora irrelevante, mientras que, por el contrario, paralelizar DCT sí que produce un cambio notorio. Además, paralelizar procesos más pequeños como ELA o algunas operaciones simples por píxel puede no compensar, porque el coste de crear hilos, sincronizarlos y repartir el trabajo puede ser mayor que el beneficio obtenido.

En el caso de `std::async`, sí se obtuvo una mejora clara al ejecutar en paralelo los procesos independientes. El tiempo aproximado secuencial era 6694 ms (más sobrecarga), mientras que con `std::async` el tiempo total con sobrecarga fue 3732 ms (ya que los 5 procesos se ejecutaron todos en paralelo). Esto supone una mejora, pero no significa que paralelizar todo sea siempre lo mejor. La mejora se debe sobre todo a que las tareas largas, como las DCT, se solapan en el tiempo. Aun así, el tiempo total no baja proporcionalmente al número de tareas, porque hay sobrecarga, competencia por memoria y limitaciones del número de núcleos disponibles.

(2) ¿Se degrada el rendimiento al paralelizar ciertas partes? (3) Si es así, ¿a qué creéis que se debe esa degradación?

Sí, el rendimiento puede degradarse al paralelizar ciertas partes, especialmente si esas partes paralelizadas tienen poco trabajo o en caso de que se creen demasiados hilos para esa tarea. En la práctica, lo que tiene más sentido paralelizar son las tareas más costosas, como en este caso DCT, en cambio las tareas como `compute_ela` y `compute_srm` de 3x3, no es tan beneficioso paralelizarlas. Si se paralelizan operaciones demasiado pequeñas, el tiempo extra de crear hilos, repartir trabajo y sincronizar resultados puede ser mayor que el ahorro conseguido.

También puede degradarse si se combinan varias estrategias a la vez sin control. Por ejemplo, si se lanzan varias tareas con `std::async` y, dentro de cada tarea, también se usan bucles OpenMP con muchos hilos, se puede producir sobresuscripción: el programa crea más hilos de los que el procesador puede ejecutar eficientemente. En ese caso, los hilos compiten por los mismos núcleos, por la caché y por el acceso a memoria, y el rendimiento puede empeorar.

(4) ¿Cuál es la mejor estrategia de paralelización?

La mejor estrategia de paralelización consiste en paralelizar principalmente las partes con mayor coste computacional y con menor dependencia entre datos. En esta práctica, la parte más costosa es el proceso de DCT por lo que el bucle que procesa los bloques de la imagen es el principal candidato para OpenMP. Cada bloque puede calcularse de forma independiente, lo que permite explotar paralelismo de datos de forma eficiente.

Además, los distintos procesos son independientes entre sí una vez cargada la imagen. Por ello, `std::async` permite explotar paralelismo funcional ejecutando estos procesos de forma concurrente.

Sin embargo, la mejor estrategia no es paralelizar sin filtro todas las partes del programa, sino combinar el paralelismo funcional entre procesos independientes con el paralelismo de datos en las partes más costosas, especialmente en `compute_dct`, controlando siempre el número de hilos para evitar sobresuscripción.

En caso de tener que elegir solo una estrategia, elegiría `std::async` para este programa en particular, ya que viendo los resultados, es el que ofrece menor tiempo total de ejecución, ya que aprovecha la ventaja de poder ejecutar simultáneamente los 5 procesos, provocando que el tiempo total sea un $\max(\text{proceso más largo})$ más la sobrecarga. No obstante, desde el punto de vista del código, la paralelización con OpenMP de la DCT es la más interesante, porque actúa directamente sobre el cuello de botella principal del programa.

Diagramas realizados

A continuación, se expondrá los diagramas de flujo que se han realizado, durante el paralelismo de código:

Diagrama de flujo Async:

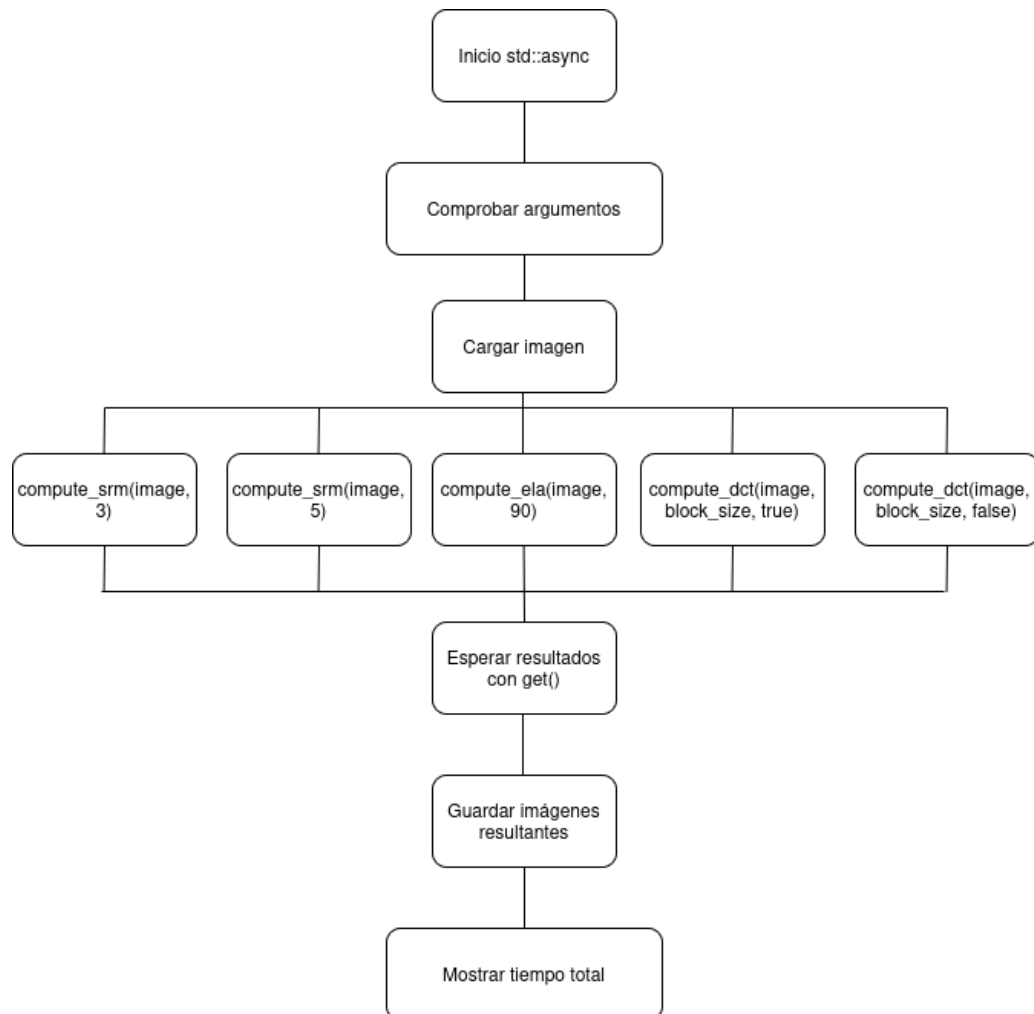
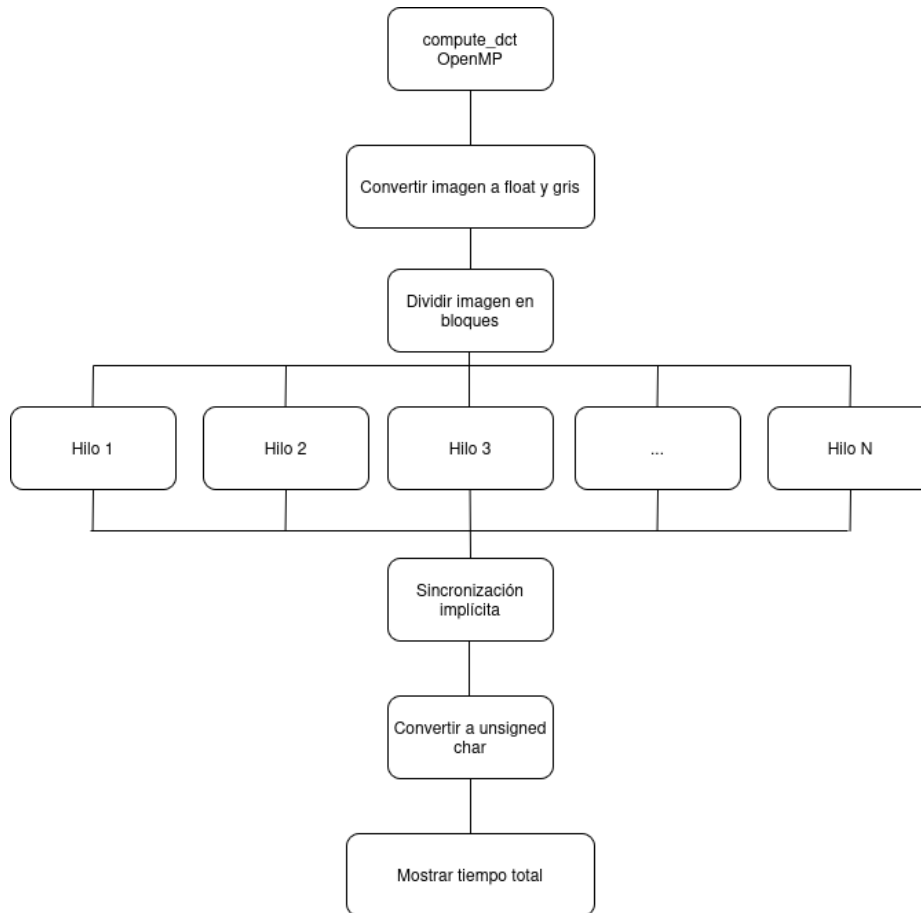


Diagrama de flujo OpenMP:



Como `async` lo que hace es que podamos paralelizar todos los procesos, representamos el flujo del `main`, mientras que, `openMP` solo lo hemos podido utilizar para optimizar las iteraciones del bucle de DCT, por lo que el diagrama de flujo, representa el flujo de la función `compute_dct`.

2.3.1 Sobre el código implementado

ASYNC:

Respecto al código original en la implementación de `std::async` la única función que varía es el `main`, copiamos su código a continuación para ir explicándolo paso por paso:

```

int main(int argc, char **argv) {
    if(argc == 1) {
        std::cerr<<"Image filename missing from arguments.
Usage ./dct <filename>"<<std::endl;

        exit(1);
    }

    auto begin_total = std::chrono::steady_clock::now();
    int block_size=8;
    Image<unsigned char> image = load_from_file(argv[1]);
    std::cout<<"Tareas asíncronas"<<std::endl;
    std::future<Image<unsigned char>> future_srm3x3 =
        std::async(std::launch::async, compute_srm,
std::cref(image), 3);

    std::future<Image<unsigned char>> future_srm5x5 =
        std::async(std::launch::async, compute_srm,
std::cref(image), 5);

    std::future<Image<unsigned char>> future_ela =
        std::async(std::launch::async, compute_ela,
std::cref(image), 90);

    std::future<Image<unsigned char>> future_dct_invert =
        std::async(std::launch::async, compute_dct,
std::cref(image), block_size, true);

    std::future<Image<unsigned char>> future_dct_direct =
        std::async(std::launch::async, compute_dct,
std::cref(image), block_size, false);

    Image<unsigned char> srm3x3 = future_srm3x3.get();
    Image<unsigned char> srm5x5 = future_srm5x5.get();
    Image<unsigned char> ela = future_ela.get();
    Image<unsigned char> dct_invert = future_dct_in-
vert.get();
    Image<unsigned char> dct_direct = future_dct_di-
rect.get();
    save_to_file("async_srm_kernel_3x3.png", srm3x3);
    save_to_file("async_srm_kernel_5x5.png", srm5x5);
    save_to_file("async_ela.png", ela);
    save_to_file("async_dct_invert.png", dct_invert);

```

```

    save_to_file("async_dct_direct.png", dct_direct);
    auto end_total = std::chrono::steady_clock::now();

    std::cout<<"Tiempo total de async: "
              <<std::chrono::duration_cast<std::chrono::mi-
lliseconds>(end_total - begin_total).count()
              <<"ms"<<std::endl;

Return 0;
}

```

La validación de que se pase una imagen se mantiene igual, lo siguiente es guardar el instante inicial para medir cuánto tarda en total la versión asíncrona. Posteriormente definimos el `block_size` que será de 8 y cargamos la imagen. Con `cout` ponemos un mensaje informativo que se imprimirá antes de que se imprima el nombre de la primera tarea. Lo siguiente es comenzar a lanzar las tareas asíncronas. Cada llamada a `std::async` lanza una función para que pueda ejecutarse de forma concurrente con las demás.

Primero lanzamos en paralelo `compute_srm(image, 3)` y este resultado no se obtiene inmediatamente sino que queda asociado a `future_srm3x3`, un future es como una “promesa de resultado”: el cálculo está ejecutándose y más adelante se podrá recoger su resultado con `.get()`. Posteriormente, lanzamos en paralelo de la misma manera el resto de procesos, siguiendo con `compute_srm(image,5)`, `compute_ela(image, 90)`, `compute_dct(image,8,true)` y `compute_dct(image,8,false)`.

Una vez lanzadas todas las tareas recogemos los resultados con `future_nombre_proceso.get()` y los vamos almacenando en variables, que posteriormente guardaremos. Por último, guardamos el instante final, e imprimimos el tiempo total de ejecución con la resta entre el instante final y el inicial.

OPENMP:

Los principales cambios para la paralelización con OpenMP se realizan en la función `compute_dct`, luego también se hacen cambios en el `main` pero son muy ligeros y también se explicarán una vez explicado el `compute_dct`, el código es:

```

Image<unsigned char> compute_dct(const Image<unsigned
char> &image, int block_size, bool invert) {
    auto begin = std::chrono::steady_clock::now();
    std::cout<<"Computing DCT ";
    if (invert) std::cout<<" inverse";
    else std::cout<<" direct";
    std::cout<<" DCT "<<block_size<<"x"<<block_size<<"
with OpenMP..."<<std::endl;

```

```

Image<float> grayscale = image.con-
vert<float>().to_grayscale();
std::vector<Block<float>> blocks = grayscale.get_blo-
cks(block_size);

#pragma omp parallel for schedule(dynamic)
for(int i = 0; i < blocks.size(); i++){
    float **dctBlock = dct::create_matrix(block_size,
block_size);

    dct::direct(dctBlock, blocks[i], 0);

    if (invert) {
        for(int k = 0; k < blocks[i].size / 2; k++)
            for(int l = 0; l < blocks[i].size / 2;
l++)
                dctBlock[k][l] = 0.0;

        dct::inverse(blocks[i], dctBlock, 0, 0.0,
255.);
    } else {
        dct::assign(dctBlock, blocks[i], 0);
    }

    dct::delete_matrix(dctBlock);
}

```

Comenzamos midiendo el instante inicial para posteriormente poder calcular el tiempo de ejecución de este proceso, posteriormente ponemos un mensaje informativo con cout, y dependiendo del valor de inverse, imprimimos inverse o direct. Al igual que la implementación original, se convierte a float y a escala de grises y posteriormente se realiza la división en bloques. Ahora llegamos al bucle, donde aparece ya OpenMP.

Con #pragma omp parallel for schedule(dynamic), dividimos las iteraciones del bucle en los diferentes hilos, cada iteración trabaja con un bloque distinto, además, hemos definidos que el reparto se haga de manera dinámica, lo que significa que OpenMP va dando bloques a los hilos conforme van terminando. Esto puede ser útil si no todos los bloques tardan exactamente lo mismo en procesarse. El resto del contenido del bucle for está exactamente igual al original, por lo que nos vamos a saltar el resto de explicación.

Además, en el main se han añadido tres líneas más respecto al original, de manera que, imprimimos cuántos hilos se utilizan con `omp_get_max_threads()`, capturamos el instante de tiempo inicial, el final e imprimimos el tiempo total de ejecución de todo el programa con la resta entre ambos.

2.3.2 Sobre el paralelismo explotado

Tipo de paralelismo usado:

En la práctica se han utilizado principalmente dos tipos de paralelismo: paralelismo funcional y paralelismo de datos.

Por un lado, en la versión con `std::async` se explota paralelismo funcional, ya que se ejecutan de forma paralela distintos procesos independientes sobre la misma imagen, como estos procesos no dependen entre sí, se pueden lanzar de manera paralela como tareas independientes.

Por otro lado, OpenMP explota el paralelismo de datos, ya que se paraleliza el bucle que recorre los bloques de la imagen dentro de `compute_dct`. Cada iteración del bucle trabaja sobre un bloque distinto, por lo que las iteraciones pueden repartirse entre varios hilos mediante `#pragma omp parallel for`.

Modo de programación paralela:

Se utiliza programación paralela a nivel de hilos en memoria compartida, esto es ya que tanto OpenMP como `std::async` crean hilos dentro del mismo proceso y todos ellos trabajan sobre el mismo espacio de memoria. En la versión de OpenMP se utiliza paralelización de bucles sobre el bucle que recorre los bloques de la DCT. Por otro lado, la versión de `std::async` utiliza programación asíncrona basada en tareas, ya que cada análisis de la imagen se lanza como una tarea independiente y el resultado se recupera posteriormente mediante `future.get()`.

Alternativas de comunicación empleadas:

La comunicación utilizada es principalmente implícita mediante memoria compartida. En la versión OpenMP, los hilos comparten estructuras como la imagen en escala de grises y el vector de bloques. Cada hilo trabaja sobre distintos elementos de blocks, mientras que variables creadas dentro del bucle, como `dctBlock`, son privadas de cada iteración, los hilos tienen variables locales independientes, pero comparten datos globales y memoria dinámica.

No se utiliza comunicación explícita mediante mensajes, ni tampoco mecanismos explícitos como son `atomic` o `critical` porque el flujo del código evita que varios hilos escriban simultáneamente sobre la misma posición de memoria. La sincronización en OpenMP es implícita al final del `parallel for`: el programa no continúa hasta que todos los hilos han terminado sus iteraciones

Por otro lado, en la versión con `std::async`, la sincronización se realiza mediante los objetos `future`. Cada llamada a `.get()` espera a que la tarea correspondiente haya terminado y recupera su resultado. Por tanto, aunque no hay comunicación explícita entre tareas, sí hay sincronización cuando el hilo principal recoge los resultados.

Estilo de programación paralela utilizado:

El estilo de programación paralela empleado es estilo de variables compartidas. En la versión de OpenMP los hilos trabajan dentro del mismo proceso y acceden a estructuras compartidas en memoria, la imagen y el vector de bloques son datos compartidos, mientras que las variables locales de cada iteración, como `dctBlock`, son privadas para cada hilo. Esto evita condiciones de carrera, ya que cada hilo procesa un bloque distinto. En la versión `std::async`, también se utiliza memoria compartida porque todas las tareas leen la misma imagen original. Sin embargo, la imagen se pasa como referencia constante mediante `std::cref(image)`, por lo que las tareas solo la leen y no la modifican.

Tipo de estructura paralela del programa:

En la versión con `std::async`, la estructura es de tipo paralelismo de tareas, el hilo principal carga la imagen y después lanza varias tareas independientes. Estas tareas se ejecutan concurrentemente y posteriormente el hilo principal espera sus resultados mediante `future.get()`. En la versión OpenMP, la estructura es de tipo paralelismo de bucle, se crea una región paralela, donde se reparte las iteraciones del bucle entre los hilos y, al final del bucle, los hilos se sincronizan antes de continuar.

2.3.3 Sobre los resultados de las pruebas realizadas y su contexto

Caracterización de la máquina paralela en la que se ejecuta el programa. (p.ej. Número de nodos de cómputo, sistema de caché, tipo de memoria, etc.).

La caracterización de la máquina paralela permite describir el entorno hardware donde se han ejecutado las pruebas de rendimiento. Esta información es importante porque el número de núcleos, hilos lógicos, memoria disponible y jerarquía de caché influyen directamente en la eficiencia de la paralelización mediante OpenMP y `std::async`, tal y como se solicita en la práctica

A partir de estos datos, se puede concluir que las pruebas se han realizado sobre una máquina paralela de memoria compartida, con un único procesador físico y un único nodo NUMA. El procesador dispone de 12 núcleos físicos y 16 CPUs lógicas visibles por el sistema operativo, por lo que permite ejecutar varios hilos de forma concurrente. Además, la aparición del indicador ht en los flags de la CPU indica que el procesador soporta Hyper-Threading, es decir, la ejecución de más de un hilo hardware por núcleo. La jerarquía de caché, formada por cachés L1, L2 y L3, es relevante para el rendimiento del programa, ya que el análisis de imágenes trabaja con grandes cantidades de datos en memoria. Por último, el sistema cuenta con 15 GiB de RAM, de los cuales 9,0 GiB estaban disponibles durante la medición, por lo que dispone de memoria suficiente para ejecutar las versiones secuencial y paralela del programa sin depender principalmente de la memoria de intercambio.

La tabla es la siguiente:

Características	Valor obtenido
Sistema de memoria	Memoria compartida/ centralizada
Arquitectura	x86_64
Modos de operación de la CPU	32-bit, 64-bit
Orden de bytes	Little Endian
Fabricante	GenuineIntel
Modelo de procesador	13th Gen Intel(R) Core(TM) i5-1340P
Números de sockets	1
Núcleos físicos por socket	12
CPUs lógicas visibles	16
Hilos por núcleo	2
Frecuencia máxima	4.6 GHz
Frecuencia mínima	400 MHz
Virtualización	VT-x
Indicador ht	Sí aparece en los flags
Caché L1 de datos	448 KiB, 12 instancias
Caché L1 de instrucciones	640 KiB, 12 instancias
Caché L2	9 MiB, 6 instancias
Caché L3	12 MiB, 1 instancia
Nodos NUMA	1
CPUs del nodo NUMA 0	0-15
Memoria RAM total	15 GiB
Memoria RAM usada en el momento de la prueba	6,3 GiB
Memoria RAM libre	2,2 GiB
Memoria RAM disponibles	9,0 GiB
Memoria de intercambio,swap	4,0 GiB

¿Qué significa la palabra ht en la salida de la invocación de la orden anterior? ¿Aparece en su ordenador de prácticas?

La palabra ht que aparece en la salida, significa Hyper-Threading.

Esto indica que el procesador soporta la ejecución de varios hilos hardware por núcleo físico. Es decir, un mismo núcleo físico puede presentarse al sistema operativo como más de un procesador lógico, permitiendo ejecutar varios hilos de forma simultánea y mejorar el aprovechamiento de los recursos internos de la CPU.

Cálculo de gráficas y cálculos

Vamos a empezar comparando los diferentes resultados de tiempo y speed-up de la versión de OpenMP con número diferente de hilos y crearemos una tabla para que se vea más fácil, compararemos el tiempo y el speed-up con 1 hilo, 2 hilos, 4 hilos, 8 hilos y finalmente 16 hilos.

Número de hilos	Tiempo	Speed-up
1	5477 ms	1.00
2	3347 ms	1.64
4	2335 ms	2.35
8	1994 ms	2.75
16	1803 ms	3.04

Los cálculos del speed-up de la tabla salen de aplicar esta fórmula:

$$\text{Speed-up} = \text{Tiempo con 1 hilo} / \text{Tiempo con } N \text{ hilos} \quad (1)$$

por ejemplo, para el primer caso, tendríamos $= 5477/3347 = 1.64$.

Se observa que al aumentar el número de hilos el tiempo de ejecución disminuye y el speed-up aumenta. Sin embargo, la mejora no es lineal: al pasar de 1 a 2 hilos se obtiene una mejora notable, pero a partir de 8 y 16 hilos la ganancia adicional es menor. Esto se debe a que no todo el programa está paralelizado, existen costes de creación y sincronización de hilos, y también puede haber competencia por memoria y caché. Por tanto, aunque con 16 hilos se obtiene el mejor tiempo, el speed-up real queda lejos del ideal teórico de 16.

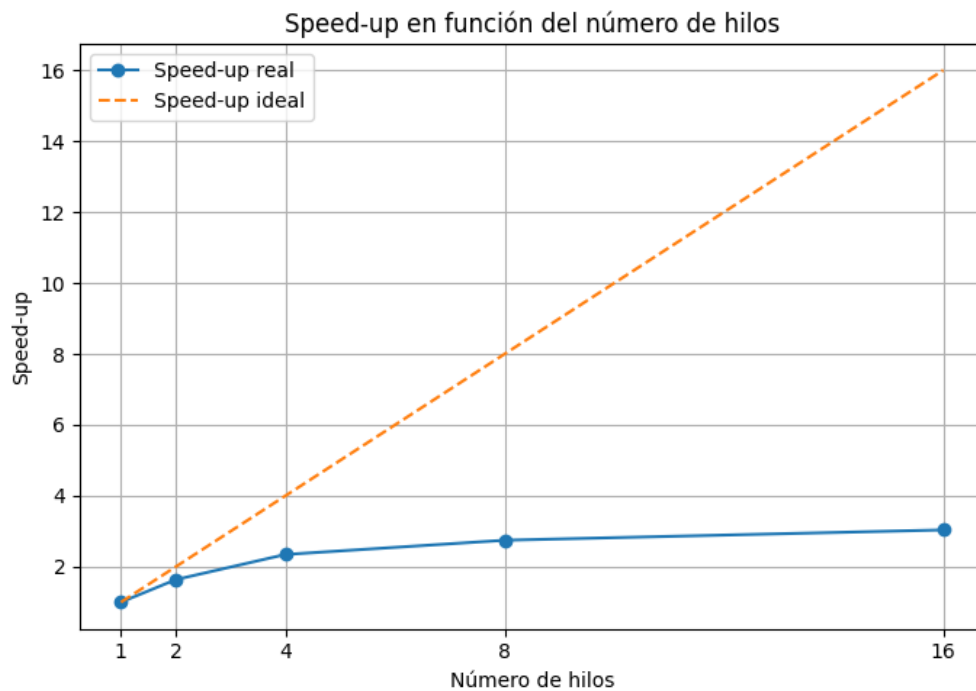
Para comparar el speed-up y tiempo cambiando el tamaño del problema, se ha decidido descargar imagemagick, que permite con `convert face_swap_enhanced.png -resize, redimensionar una imagen al tamaño que definamos, vamos a probar con varios`

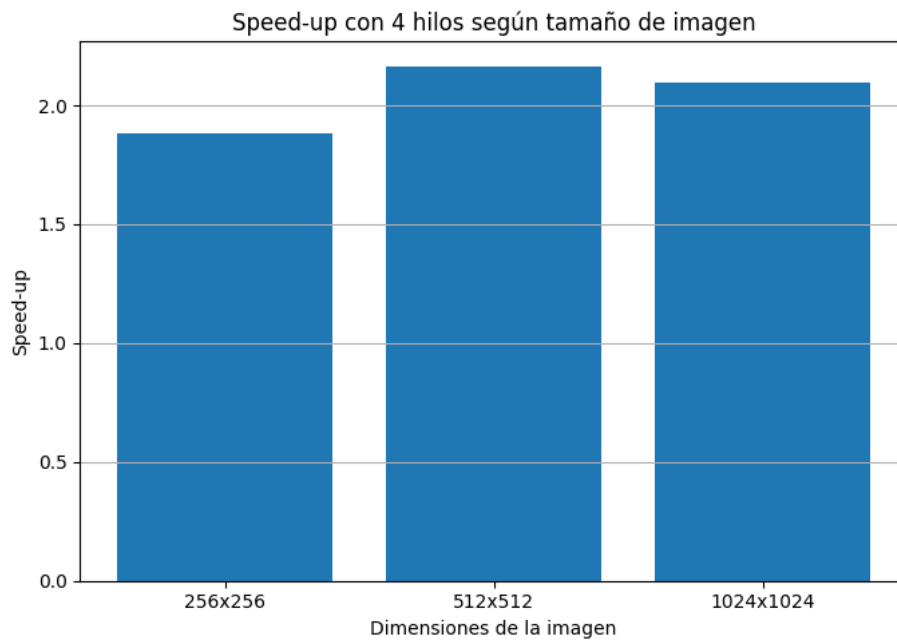
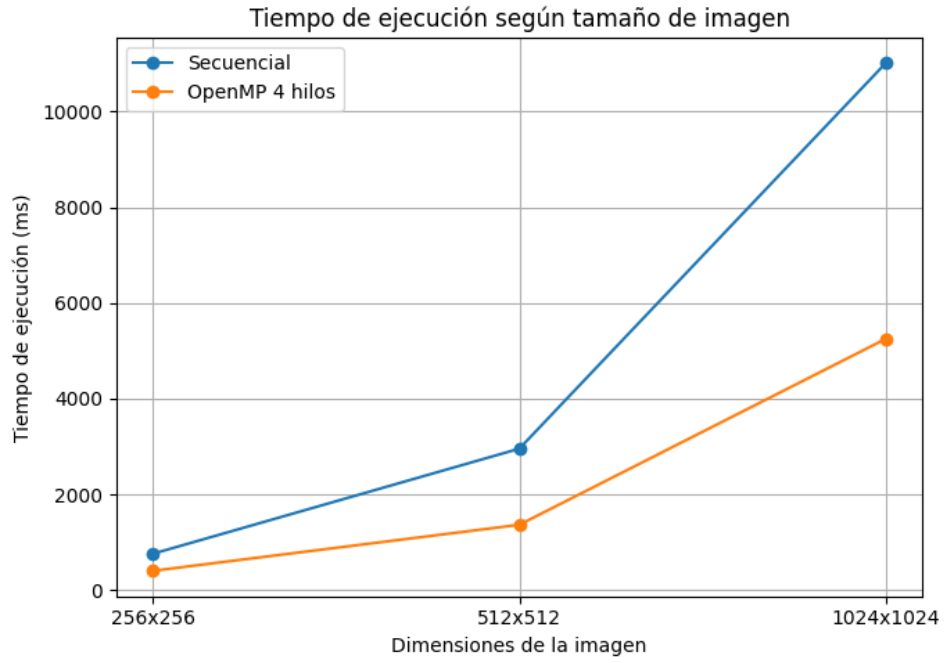
tamaños, primero con 256x256, luego con 512x512 y por último con 1024x1024. La tabla se presenta a continuación:

Dimensiones	Tiempo secuencial	Tiempo 4 hilos	Speed-up
256x256	760 ms	404 ms	1.88
512x512	2957 ms	1367 ms	2.16
1024x1024	11018 ms	5254 ms	2.10

Se observa que el uso de 4 hilos mejora el rendimiento en todos los tamaños de imagen. El speed-up se sitúa alrededor de 2x, aunque no llega al valor ideal de 4 debido a que no todo el programa está paralelizado, existen partes secuenciales y hay sobrecostes de creación, planificación y sincronización de hilos. Además, el acceso a memoria y la competencia por caché también limitan la escalabilidad.

Se han realizado dos gráficas para representar el speed-up con respecto a los hilos y el tiempo de ejecución respecto a las dimensiones de las imágenes. Además, también se ha ilustrado una gráfica de barras para representar el speed-up respecto al tamaño de la imagen.





Finalmente, para responder a la pregunta de la velocidad máxima teórica, podemos observar a partir de los datos obtenidos que la paralelización mejora el rendimiento del programa, aunque la mejora no es lineal respecto al número de hilos. Se puede ver que al aumentar el número de hilos el tiempo de ejecución disminuye. Sin embargo, la ganancia obtenida no crece proporcionalmente. Por ejemplo, con 16 hilos no se consigue un speed-up de 16, sino de aproximadamente 3.04. Esto se debe a que no todo el programa está paralelizado, lo que hace que la parte no paralelizable penalice el speed-up máximo.

La ganancia máxima teórica ideal, si todo el programa fuera perfectamente paralelizable y no existiera ninguna sobrecarga, sería igual al número de hilos utilizados, por lo que, con 4 hilos, el speed-up sería de 4, mientras que, con 16, sería de 16. Sin embargo, esta ganancia ideal no se alcanza en la práctica. La ganancia máxima real está limitada por la parte secuencial del programa, como indica la Ley de Amdahl.

¿CUAL ES LA IMPLEMENTACIÓN MÁS EFICIENTE DE LAS DOS?

Realmente, el resultado depende del número de hilos que se utilicen para la ejecución de la versión de OpenMP, ya que con 16 hilos, como hemos visto anteriormente, obtenemos un speed-up de 3.04, mientras que el speed-up fijo de `std::async` es de 1.79 aproximadamente. Sin embargo, si utilizamos solo 2 hilos, obtenemos un speed-up similar, aunque más bajo, de 1.64. Por tanto, la implementación más eficiente en nuestras pruebas es la versión OpenMP, especialmente al aumentar el número de hilos.

Esto se debe a que OpenMP actúa directamente sobre el cuello de botella principal del programa: la DCT. En los tiempos secuenciales, la DCT inversa y la DCT directa eran las partes más costosas, con 3398 ms y 2139 ms. Al paralelizar el bucle que procesa los bloques de la DCT, OpenMP reduce de forma importante el tiempo de la parte más pesada del programa. En cambio, `std::async` mejora el rendimiento ejecutando los procesos independientes a la vez, pero no reduce internamente el coste de cada proceso. Es decir, la DCT inversa sigue tardando aproximadamente lo mismo, solo que se ejecuta simultáneamente con el resto de tareas. Por eso el tiempo total de `std::async` se acerca al tiempo de la tarea más lenta más la sobrecarga.

References

1. OpenMP Architecture Review Board: OpenMP Application Programming Interface, Version 6.0. OpenMP Architecture Review Board (2024), <https://www.openmp.org/wp-content/uploads/OpenMP-API-Specification-6-0.pdf>, last accessed 2026/05/01
2. OpenMP Architecture Review Board: Worksharing-Loop Construct and schedule Clause. In: OpenMP Application Programming Interface, Version 6.0. OpenMP Architecture Review Board (2024), <https://www.openmp.org/wp-content/uploads/OpenMP-API-Specification-6-0.pdf>, last accessed 2026/05/01
3. OpenMP Architecture Review Board: shared Clause. In: OpenMP Application Programming Interface, Version 5.2. OpenMP Architecture Review Board (2021), <https://www.openmp.org/wp-content/uploads/OpenMP-API-Specification-5-2.pdf>, last accessed 2026/05/01
4. OpenMP Architecture Review Board: private Clause. In: OpenMP Application Programming Interface, Version 5.2. OpenMP Architecture Review Board (2021), <https://www.openmp.org/wp-content/uploads/OpenMP-API-Specification-5-2.pdf>, last accessed 2026/05/01
5. OpenMP Architecture Review Board: omp_get_wtime Routine. In: OpenMP Application Programming Interface, Version 6.0. OpenMP Architecture Review Board (2024), <https://www.openmp.org/wp-content/uploads/OpenMP-API-Specification-6-0.pdf>, last accessed 2026/05/01
6. Grama, A., Gupta, A., Karypis, G., Kumar, V.: Introduction to Parallel Computing. 2nd edn. Addison-Wesley, Boston (2003)
7. cppreference.com: std::async. <https://en.cppreference.com/w/cpp/thread/async>, last accessed 2026/05/01
8. cppreference.com: std::future::wait. <https://en.cppreference.com/w/cpp/thread/future/wait>, last accessed 2026/05/01
9. cppreference.com: std::future::get. <https://en.cppreference.com/w/cpp/thread/future/get>, last accessed 2026/05/01
10. cppreference.com: std::thread. <https://en.cppreference.com/w/cpp/thread/thread>, last accessed 2026/05/01
11. cppreference.com: std::vector. <https://en.cppreference.com/w/cpp/container/vector>, last accessed 2026/05/01
12. cppreference.com: std::vector::push_back. https://en.cppreference.com/w/cpp/container/vector/push_back, last accessed 2026/05/01